SYMPRAXIS CONSULTING

Harnessing Change Events in Microsoft 365 for Enterprise-Scale Workflow Management

Julie Turner

SYMPRAXIS CONSULTING



julieturner.net/me

Julie Turner

Partner / CTO · Sympraxis Consulting

- Working with SharePoint since **2007**
- **Microsoft MVP**, Office Apps & Services since 2017
- M365 & Power Platform **Community Team** since 2019
- Open-source co-maintainer: **PnPjs** & **hTWOo**
- Co-host: ***Code. Deploy. Go Live.***

Agenda



What are webhooks and why use them



Architectural pattern for managing subscriptions and events



Cloud Services Overview



Boilerplate code walkthrough



Event handlers

What are Webhooks/Change Events?

…altering the behavior of a web page or web application with custom callbacks.



In Microsoft 365 – SharePoint “webhooks” are supported for list items.

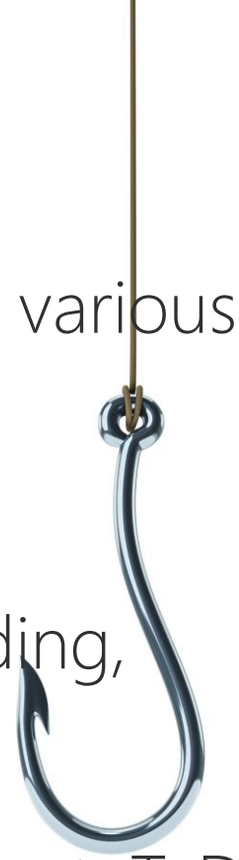
Lists/Libraries

- Expires in 180 days
- Only handles async events (-ed)
- ItemAdded, ItemUpdated, ItemDeleted, ItemCheckedOut, ItemCheckedIn, ItemUncheckedOut, ItemAttachmentAdded, ItemAttachmentDeleted, ItemFileMoved, ItemVersionDeleted, ItemFileConverted

What are Webhooks/Change Events?

In Microsoft 365 – Microsoft Graph Change Events are supported for various objects. Object type indicates limitations.

- Cloud printing, Copilot AI Interactions, OneDrive Items, Groups, SharePoint Lists, Group Conversations, Outlook Messages, Events, Contacts, Security alerts, Teams Approvals, Call Record, Call Recording, Call Transcript, Teams Chat, Chat Message, Teams Channel, Conversation Member, Online Meeting, Presence, Teams Team, Teams Shift, Offer Requests, Shift Change Requests, Time Off Request, ToDo Task, User, SharePoint Sites*
- Many resources have limits or quotas on how many subscriptions can be made against that resource.



* Not documented but is supported.

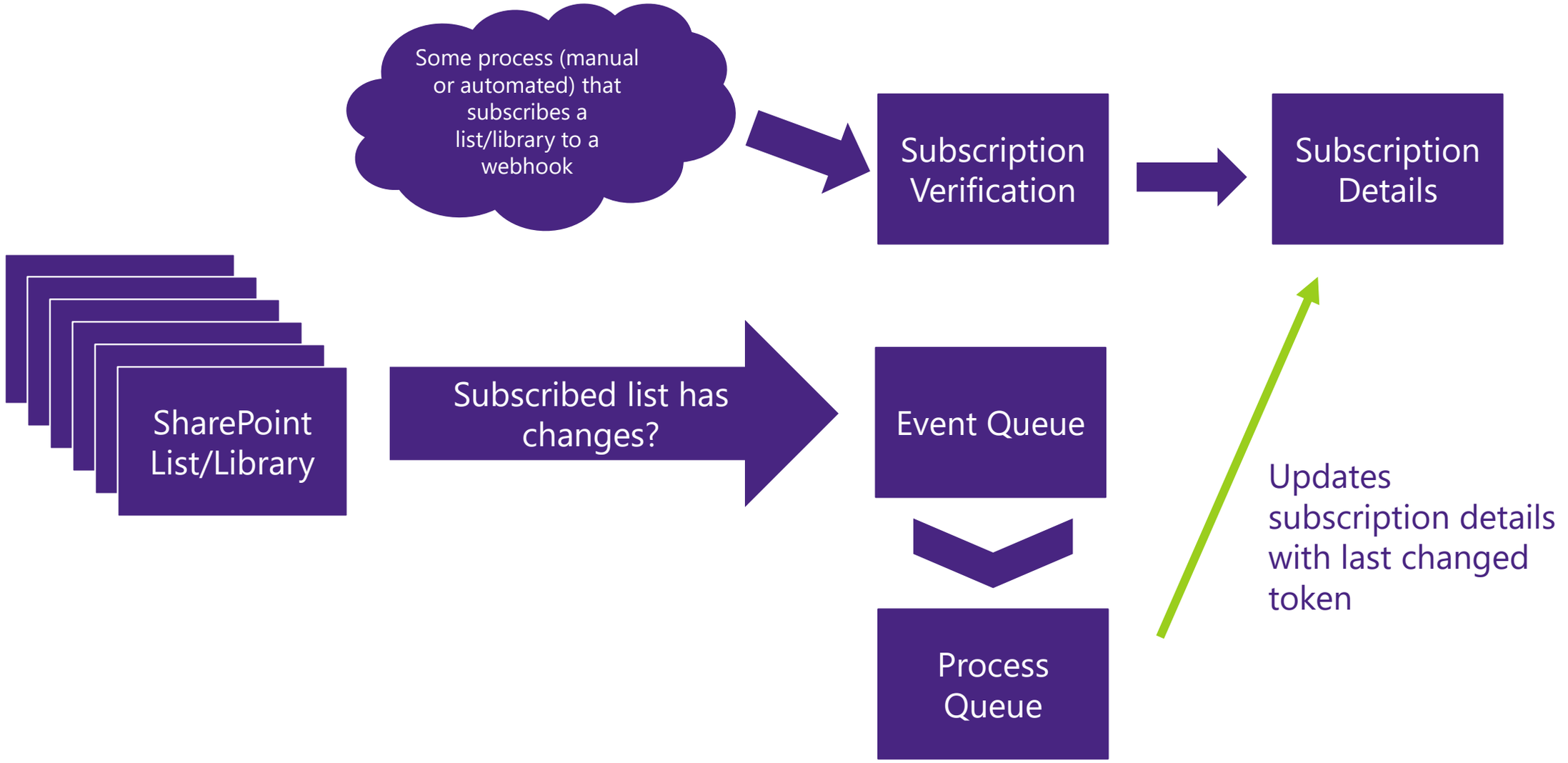
Why use Webhooks/Change Events?



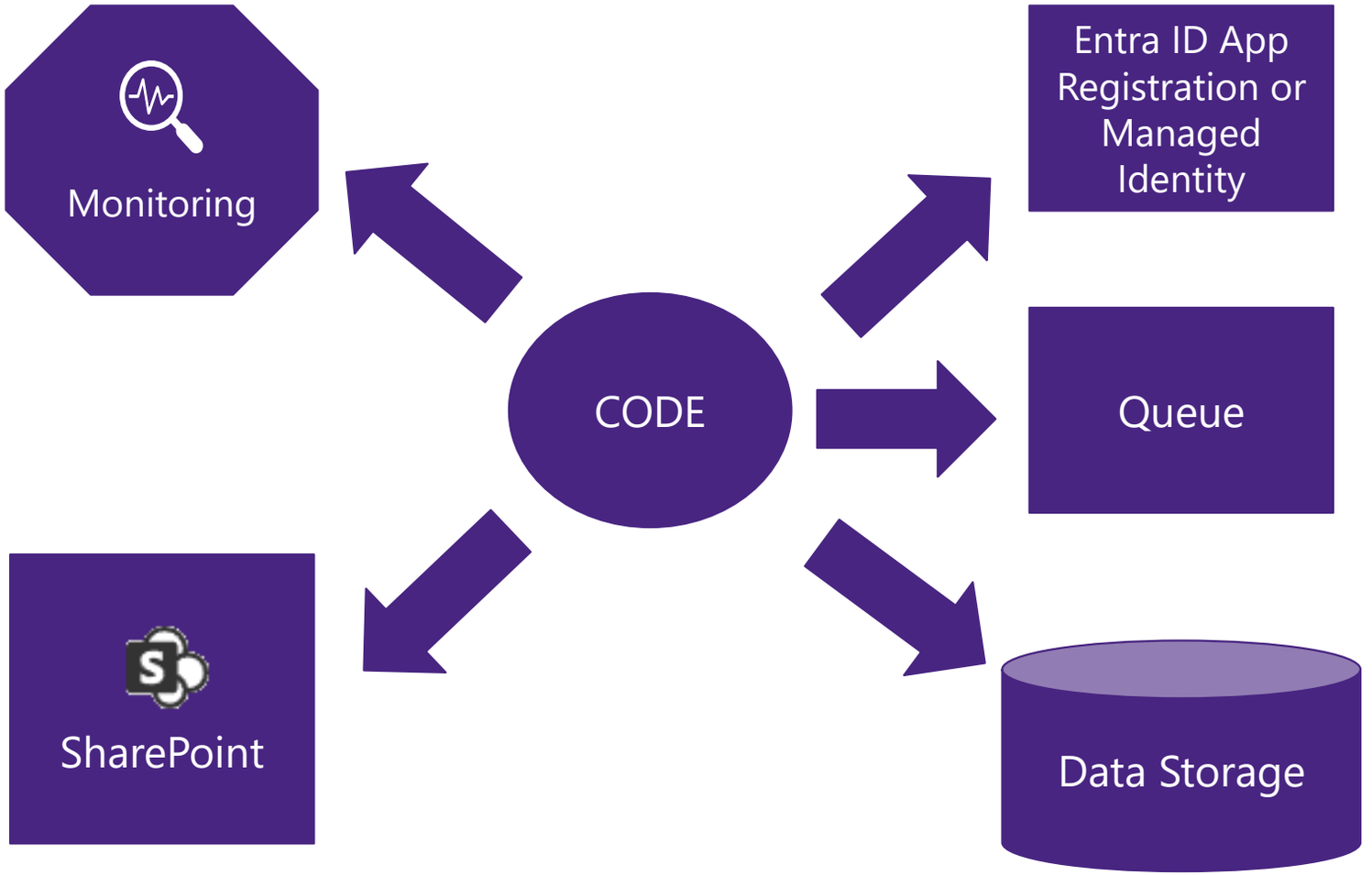
- Other Options: PowerAutomate flows, Azure LogicApps
  - PowerAutomate and LogicApps SharePoint connector is in user context
  - You must manage multiple instances for each object that you want to monitor
  - Actual monitoring of success/failure built into the product

- Webhooks/Change Events are a good option:
  - Complex business process
  - Requires application vs user context
  - Need to act on multiple objects with the same/similar logic

Process


Some process (manual or automated) that subscribes a list/library to a webhook
Subscription Verification
Subscription Details
SharePoint List/Library
Subscribed list has changes?
Event Queue
Process Queue
Updates subscription details with last changed token

Architectural Overview



Provides Application access to SharePoint

Library change notifications are written to the queue to be processed

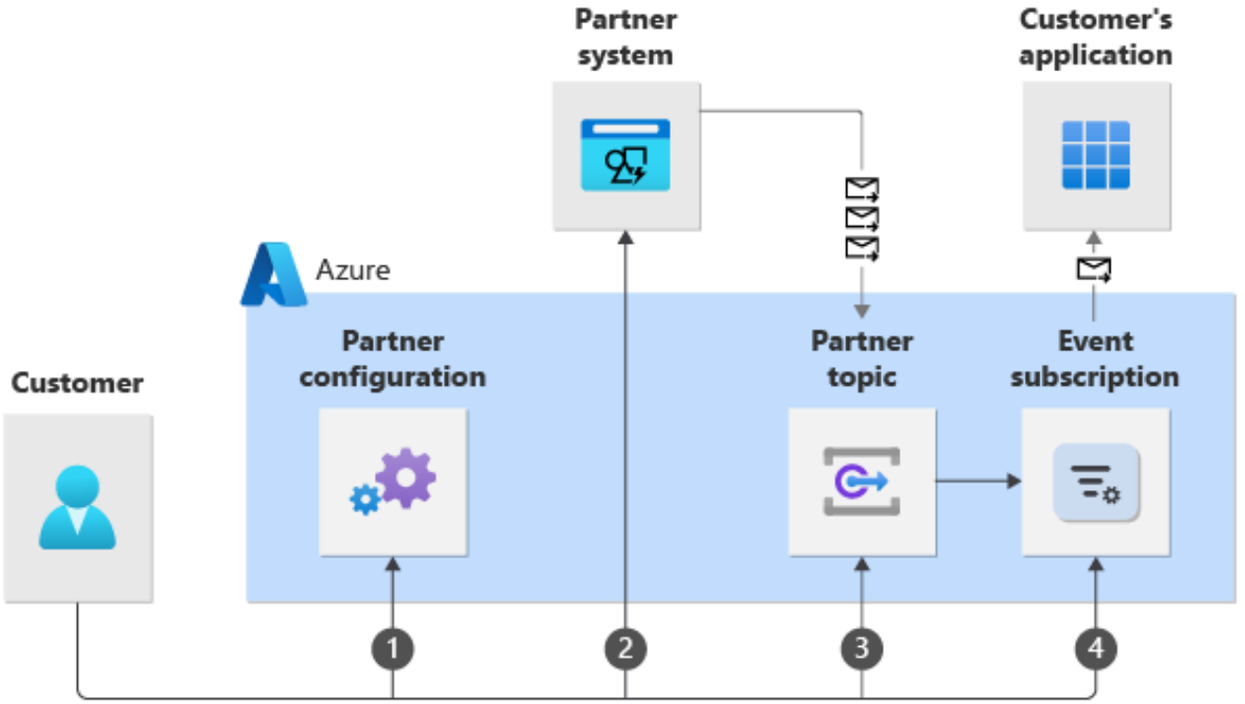
Site Collection Subscriptions
List Webhook Subscriptions
Last Change Token for Webhook

Cloud Services - Azure

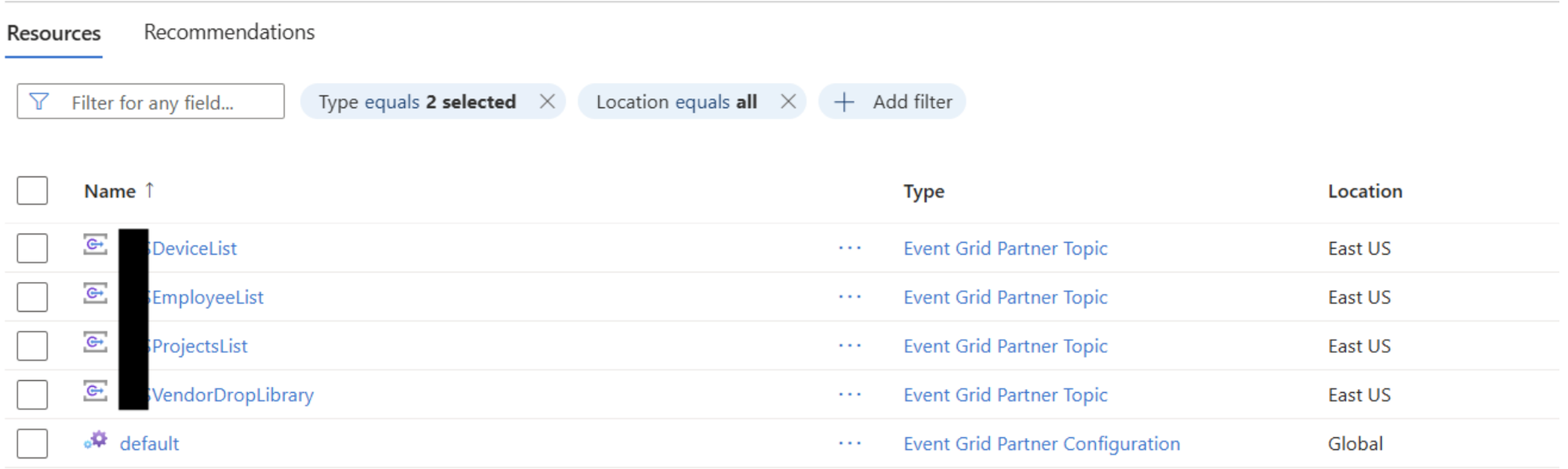
Service	Used for
Resource Group	A container for all your Azure resources related to this solution
Azure Storage	Queue store to manage event notifications and subscription info
Key Vault	Stores secrets and certificates
Function App	Code to manage subscription and execute custom event handlers
AAD App Registrations	Provides app credentials for the function apps.
Application Insights (Opt)	Provides diagnostic logging support
Azure Event Grid	Provides relay for change events

Event Grid

1. Authorize partner (Microsoft Graph) to create a partner topic in a resource group you designate.
2. Partner provisions a partner topic in the specified resource group of your Azure subscription (MyListSub).
3. Activate the partner topic.
4. Subscribe to events by creating one or more event subscriptions for the partner topic.



Event Grid Partner Configurations & Topics



Security

- EntraId App Registrations/ MSAL
- Managed Identity/ Azure Identity

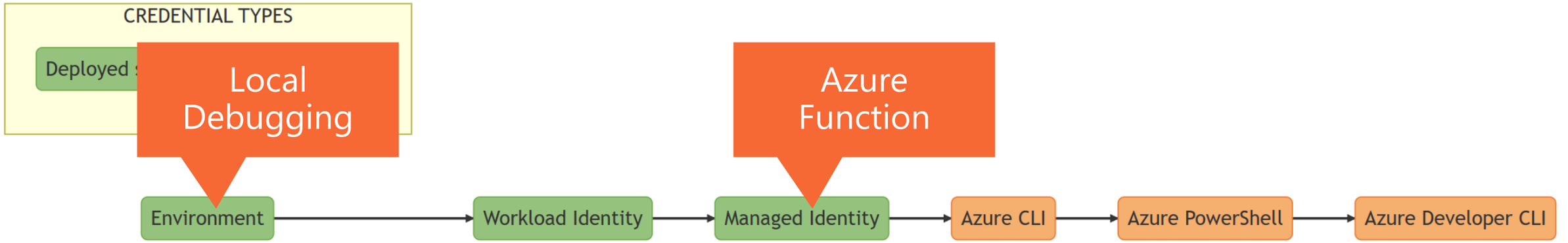


SYMPRAXIS
CONSULTING

@Azure/Identity

DefaultAzureCredential

- Environment - account information specified via environment variables and use it to authenticate.
- Workload Identity - deployed to Azure Kubernetes Service with Managed Identity enabled
- Managed Identity - deployed to an Azure host with Managed Identity enabled
- Azure CLI - developer has authenticated an account via the Azure CLI az login command
- Azure PowerShell - developer has authenticated using the Azure PowerShell module Connect-AzAccount command
- Azure Developer CLI - developer has authenticated an account via the Azure Developer CLI azd auth login comman

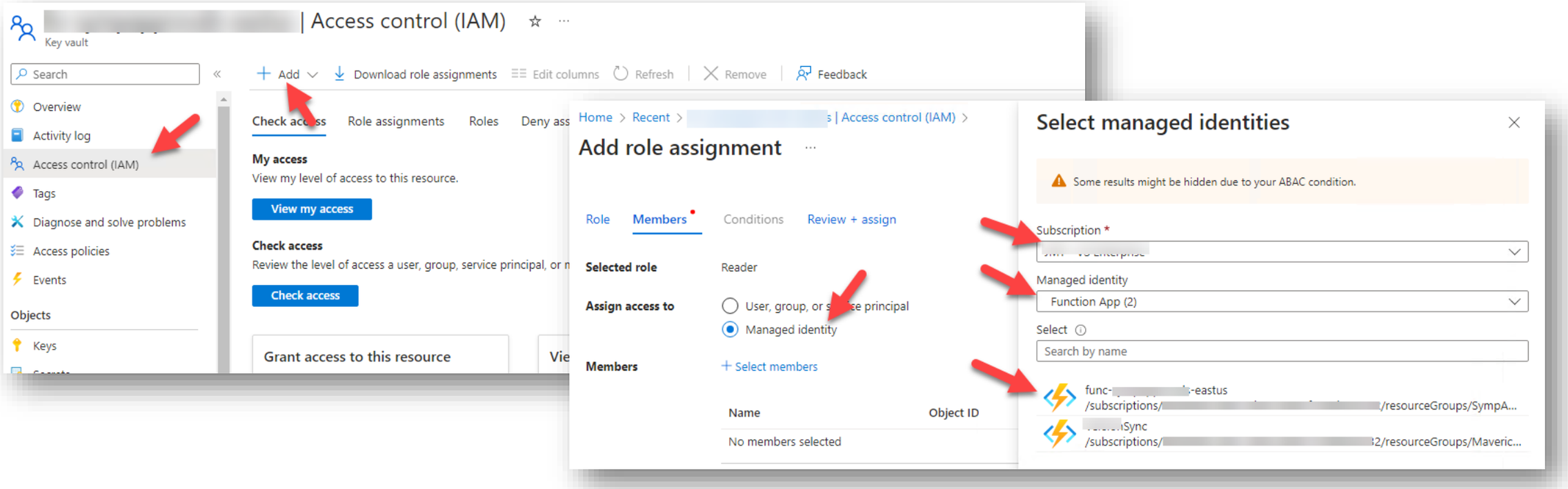


Managed Identity

Managed identities in Azure are a service that allows Azure resources to authenticate cloud services without the need for storing credentials in code or configuration files.

- You don't need to manage credentials.
  - Credentials aren’t even accessible to you.
- You can use managed identities to authenticate to any resource that supports Microsoft Entra authentication, including your own applications.
- Managed identities can be used at no extra cost.
- System-assigned
  - 1:1 relationship with the azure resource and its lifecycle is tied to the resource
- User-assigned
  - 1:many relationship to azure resources
- You authorize a managed identity to have access to one or more services

Authorizing a managed identity




```
m365 login --authType browser

m365 aad approleassignment add --appObjectId "1022615c-4433-4731-a933-53a9d2770e76" --resource "Microsoft Graph" --scopes "Files.ReadWrite.All,Group.
Read.All,Mail.Send,User.Read.All"

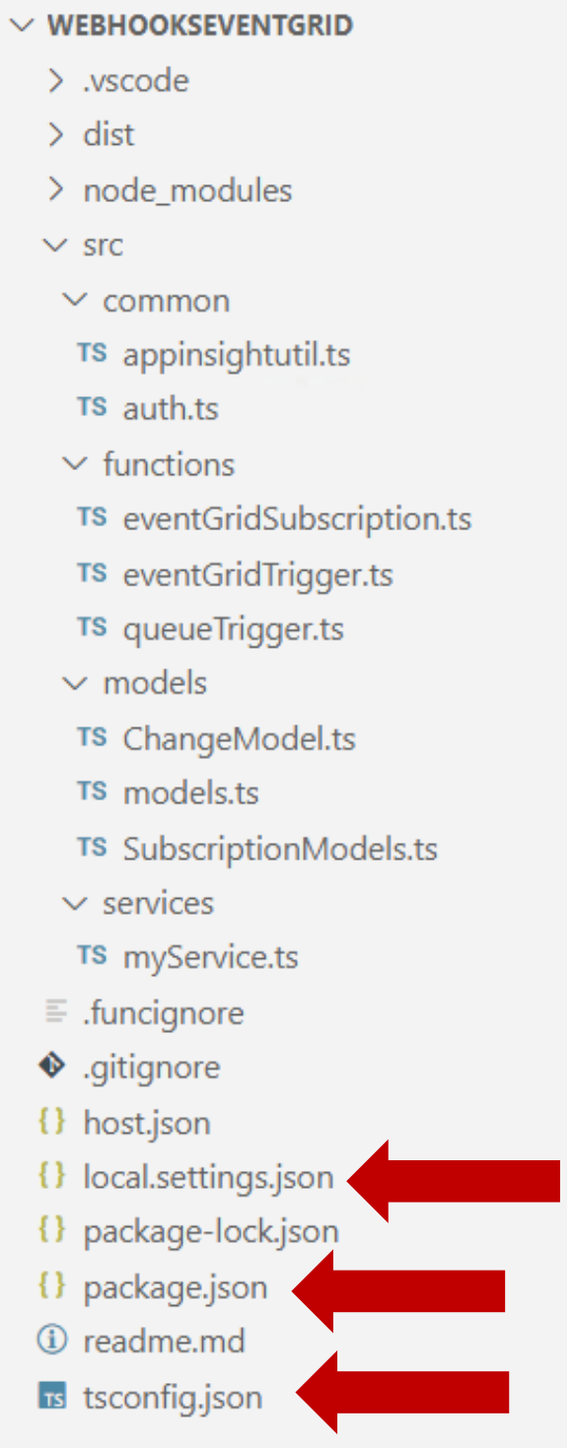
m365 aad approleassignment add --appObjectId "1022615c-4433-4731-a933-53a9d2770e76" --resource "SharePoint" --scopes "Sites.FullControl.All"

m365 aad approleassignment add --appObjectId "1022615c-4433-4731-a933-53a9d2770e76" --resource "SharePoint" --scopes "TermStore.ReadWrite.All"
```

Boilerplate Webhook Implementation

Project Structure & Dependencies

Project Folder



src	Source Code folder for solution
local.settings.json	Local debug settings for Azure Function
package.json	Package dependencies for the project
tsconfig.json	TypeScript configuration for transpiling

package.json

{} package.json > ...

```
{
  "name": "event-grid-test",
  "version": "1.0.0",
  "description": "",
  "type": "module",
  ▷ Debug
  "scripts": {
    "build": "tsc",
    "watch": "tsc -w",
    "clean": "rimraf dist",
    "prestart": "npm run clean && npm run build",
    "start": "func start",
    "test": "echo \"No tests yet...\""
  },
  "dependencies": {
    "@azure/functions": "4.16.0",
    "@pnp/azidjsclient": "4.20.0",
    "@pnp/nodejs": "4.20.0",
    "@pnp/sp": "4.20.0",
    "@pnp/graph": "4.20.0",
    "applicationinsights": "2.x"
  },
  "devDependencies": {
    "@types/node": "20.x",
    "rimraf": "6.x",
    "typescript": "5.x"
  },
  "main": "dist/src/functions/*.js"
}
```

tsconfig.json

tsconfig.json > ...

```
{
  "compilerOptions": {
    "module": "ESNext",
    "target": "ESNext",
    "outDir": "dist",
    "rootDir": ".",
    "sourceMap": true,
    "strict": false,
    "moduleResolution": "bundler",
    "allowSyntheticDefaultImports": true,
    "typeRoots": [
      "./node_modules/@types"
    ],
  }
}
```

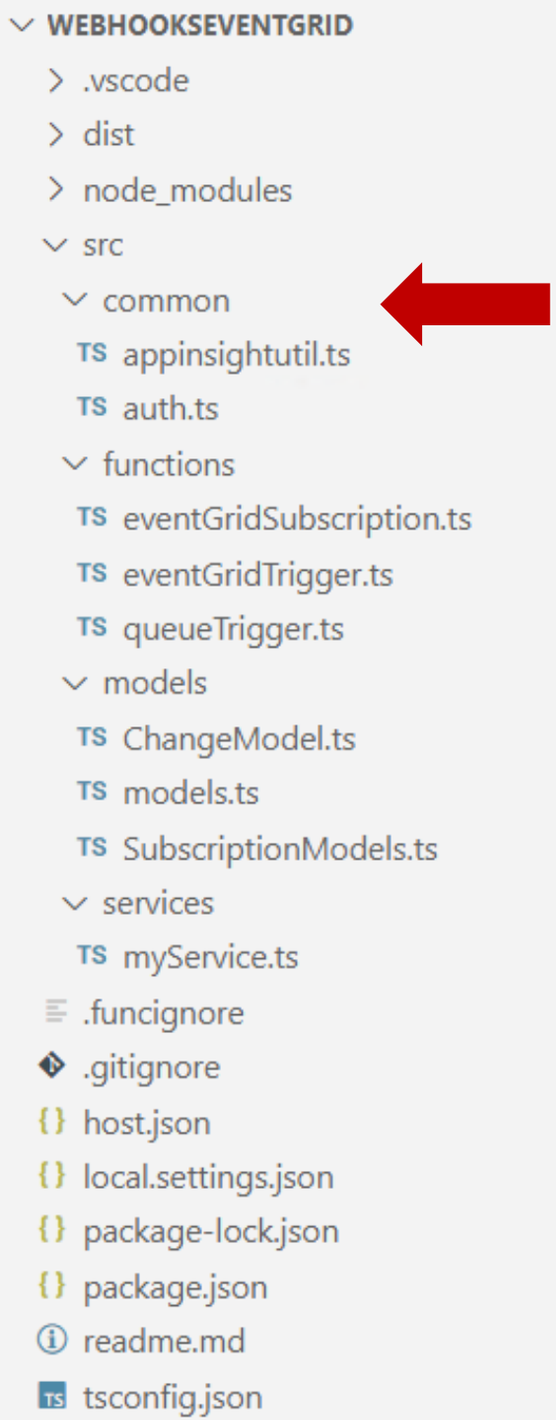
local.settings.json

```
{
  "IsEncrypted": false,
  "Values": {
    "APP_NAME": "WebhooksEventGrid",
    "APP_VERSION": "1.0.0",
    "APPINSIGHTS_LOGLEVEL": "0",
    "APPLICATIONINSIGHTS_CONNECTION_STRING":
    "InstrumentationKey=99999999-9999-9999-9999-999999999999;IngestionEndpoint=https://eastus-8.in.
    applicationinsights.azure.com/;LiveEndpoint=https://eastus.livediagnostics.monitor.azure.com/",
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "node",
    "AzureWebJobsFeatureFlags": "EnableWorkerIndexing",
    "DEBUG": true,
    "SubscriptionQueuePath": "SubscriptionQueuePath",
    "NotificationQueuePath": "webhooksEventGrid",
    "SubscriptionFile": "webhooksEventGrid/webhooksEventGrid-config.json",
    "SCHEDULE": "0 */5 * * * *",
    "Tenant": "contoso",
    "ProjectSite": "https://contoso.sharepoint.com/sites/MySite",
    "ProjectListId": "99999999-9999-9999-9999-999999999999",
    "AZURE_TENANT_ID": "99999999-9999-9999-9999-999999999999",
    "AZURE_CLIENT_ID": "99999999-9999-9999-9999-999999999999",
    "AZURE_CLIENT_CERTIFICATE_PATH": "C:\\private-key.pem"
  }
}
```

Boilerplate Webhook Implementation

Helper Classes

Project Folder



common	Helper class folder for solution
appinsightsutil.ts	Helper class to configure and log to Application Insights
auth.ts	Authentication helper class to create authenticated endpoints for PnPjs and other services

appinsightsutil.ts

```
27  export class AppInsightUtil implements IAppInsightUtil {
28    private _uGuid = getGUID();
29    private _commonProps = {};
30
31    public Init(invocationId: string): void {
32      this._initDefaultConfig(invocationId);
33      AppInsights.start();
34    }
35
36    public AddCommonProperty(keyName: string, value: string): void {
37      this._commonProps[keyName] = value;
38      AppInsights.defaultClient.commonProperties = this._commonProps;
39    }
40
41  >   public Log(type: MessageType, msg: IAppInsightMessage): void {···
59    }
60
61    /**
62   * Setup AppInsights with common settings
63   *
64   * @export
65   */
66    private _initDefaultConfig(invocationId: string): void {
67      /* check if AppInsights already configured...
68       *    > determined if `defaultClient` set & if `app.name` set
69       *    > if not, bail on the config (don't double setup)
70       */
71      if (AppInsights.defaultClient?.commonProperties['app.name']) {
72        return;
73      }
74
75      // enable stream of of live metrics; set auto dependency correlation
76      AppInsights.setup(process.env.APPLICATIONINSIGHTS_CONNECTION_STRING).setSendLiveMetrics(true).setAutoDependencyCorrelation(true);
77
78      // set the current tagged app & version of the function app
79      this._commonProps["app.name"] = process.env.APP_NAME;
80      AppInsights.defaultClient.commonProperties = this._commonProps;
81      AppInsights.defaultClient.context.tags[AppInsights.defaultClient.context.keys.applicationVersion] = process.env.APP_VERSION;
82
83      // set the operation id to the current invocation id
84      AppInsights.defaultClient.context.tags[AppInsights.defaultClient.context.keys.operationId] = `${invocationId}-${this._uGuid.toString()}`;
85
86    }
87  }
```

appinsightsutil.ts

```
4   export const SeverityLevel = AppInsights.Contracts.SeverityLevel;
5   export enum MessageType {
6     "Event",
7     "Exception",
8     "Metric",
9     "Trace"
10  }
11
12  export interface IAppInsightMessage {
13    logSource: string;
14    message?: string;
15    severity?: AppInsights.Contracts.SeverityLevel;
16    properties?: any,
17    name?: string,
18    exception?: Error
19  }
```





```
41    public Log(type: MessageType, msg: IAppInsightMessage): void {
42      if (AppInsights.defaultClient != undefined) {
43        if (msg.properties == null) { msg.properties = {}; }
          switch (type) {
45          case MessageType.Trace:
46            AppInsights.defaultClient.trackTrace({ message: msg.message, severity: msg.severity, properties: { source: msg.logSource, ...msg.properties } });
47            break;
48          case MessageType.Event:
49            AppInsights.defaultClient.trackEvent({ name: msg.logSource, properties: msg.properties });
50            break;
51          case MessageType.Exception:
52            AppInsights.defaultClient.trackException({ exception: msg.exception, severity: msg.severity, properties: { source: msg.logSource, ...msg.properties } });
53            break;
54          case MessageType.Metric:
55            AppInsights.defaultClient.trackMetric({ name: msg.logSource, value: +msg.message });
56            break;
57        }
58      }
59    }
```

auth.ts

```
1   import { DefaultAzureCredential } from "@azure/identity";
2   import { AzureIdentity } from "@pnp/azidjsclient";
3   import { GraphDefault, SPDefault } from "@pnp/nodejs";
4   import { spfi, SPFI } from "@pnp/sp";
5   import { graphfi, GraphFI } from '@pnp/graph';
6   import { IAppInsightUtil, MessageType, SeverityLevel } from "./appinsightutil.js";
7
8   export interface IAuthService {···
14  }
15
16  export class AuthService implements IAuthService {
17    private LOG_SOURCE = "AuthService";
18    private _ready: boolean = false;
19    private _sp: SPFI = null;
20    private _graph: GraphFI = null;
21    private _spAdmin: SPFI = null;
22    private _apu: IAppInsightUtil = null;
23
24    public constructor(apu: IAppInsightUtil) {
25      this._apu = apu;
26    }
27
28    public async Init(): Promise<boolean> {···
59    }
60
61    public get ready(): boolean {
62      return this._ready;
63    }
64    public get sp(): SPFI {
65      return this._sp;
66    }
67    public get graph(): GraphFI {
68      return this._graph;
69    }
70
71    public get spAdmin(): SPFI {
72      return this._spAdmin
73    }
74  }
```

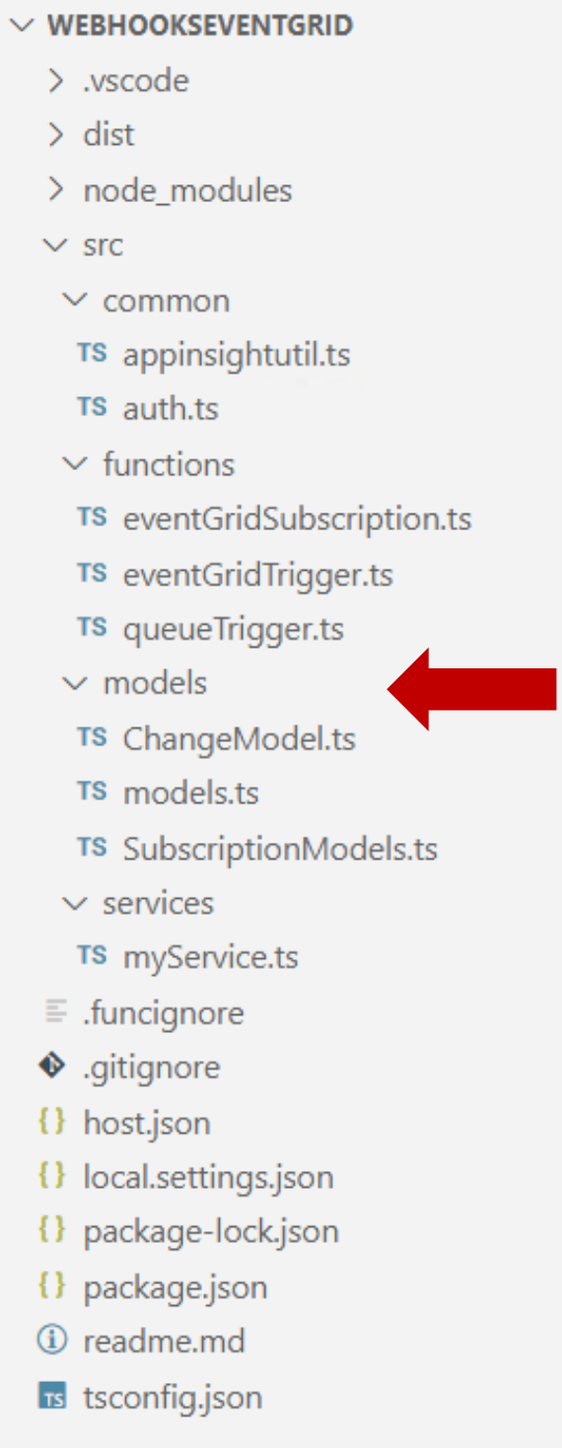
auth.ts

```
28  public async Init(): Promise<boolean> {
29    let retVal = false;
30    try {
31      const credential = new DefaultAzureCredential();
32      // Create SharePoint credentials
33      this._sp = spfi(process.env.ProjectSite).using(SPDefault(),
34        AzureIdentity(credential, [`https://${process.env.Tenant}.sharepoint.com/.default`], null));
35      // Create Graph credentials
36      this._graph = graphfi().using(GraphDefault(), AzureIdentity(credential, [`https://graph.microsoft.com/.default`], null));
37      // Create SharePoint Admin credentials
38      // const tenantUrl = `https://${process.env.Tenant}-admin.sharepoint.com`;
39      // this._spAdmin = spfi(tenantUrl).using(SPDefault(),
40      // AzureIdentity(credential, [`https://${process.env.Tenant}-admin.sharepoint.com/.default`], null));
41      this._ready = true;
42      this._apu.Log(MessageType.Trace, {
43        message: "Init success",
44        logSource: this.LOG_SOURCE,
45        properties: {
46          method: "Init"
47        },
48        severity: SeverityLevel.Verbose
49      });
50      retVal = true;
51    } catch (err) {
52      this._apu.Log(MessageType.Exception, {
53        logSource: this.LOG_SOURCE,
54        exception: err,
55        severity: SeverityLevel.Critical,
56        properties: {
57          method: "Init"
58        }
59      });
60    }
61    return retVal;
62  }
```

Boilerplate Webhook Implementation

Models

Project Folder



models	TypeScript interfaces folder for solution
ChangeModel.ts	Interface for changes from a SharePoint list
SubscriptionModel.ts	Interfaces for Event Grid subscriptions and events

SubscriptionModel.ts

```typescript
export interface SubscriptionConfig {
  subscriptionId: string;
  expirationDateTime: string;
  lastToken: string;
  graphResource: string;
}


/**
 * Example Data
 * odata.type: "#Microsoft.Graph.Group"
 * odata.id: "Groups/cf653542-be15-4568-87d4-37613d8a21f5"
 * organizationId: same as tenant id
 */

export interface IEGResourceData {···
}

/**
 * Example Data
 * changeType: "updated"
 * resource: "Groups/cf653542-be15-4568-87d4-37613d8a21f5"
 */
export interface IEGData {···
}


/**
 * Example Data
 * type: "Microsoft.Graph.ListItemUpdated"
 * source: "/tenants/46474cf3-2a42-4ac0-a447-bc5997157660/applications/de8bc8b5-d9f9-48b1-a8ad-b748da725064"
 * subject: "Groups/cf653542-be15-4568-87d4-37613d8a21f5"
 */
export interface IEGNotification {
  type: string;
  specversion: string;
  source: string;
  subject: string;
  id: string;
  time: string;
  datacontenttype: string;
  data: IEGData;
}

```

ChangeModel.ts

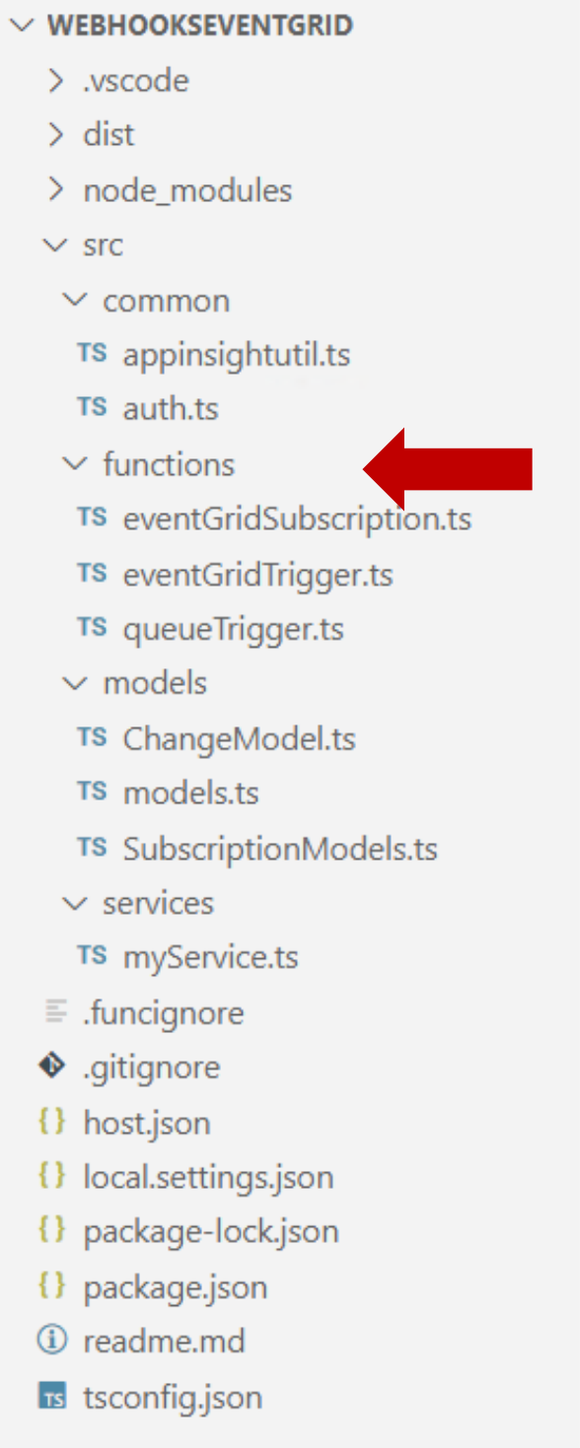
```typescript
//https://learn.microsoft.com/en-us/previous-versions/office/sharepoint-server/ee543793(v=office.15)
export enum ChangeType {
    NoChange = 0,
    Add = 1,
    Update = 2,
    DeleteObject = 3,
    Rename = 4,
    MoveAway = 5,
    MoveInto = 6,
    Restore = 7,
    RoleAdd = 8,
    RoleDelete = 9,
    RoleUpdate = 10,
    AssignmentAdd = 11,
    AssignmentDelete  = 12,
    MemberAdd = 13,
    MemberDelete = 14,
    SystemUpdate = 15,
    Navigation = 16,
    ScopeAdd = 17,
    ScopeDelete = 18,
    ListContentTypeAdd = 19,
    ListContentTypeDelete = 20,
    Dirty = 21,
    Activity = 22,
};

export interface IChangeModel {
    "odata.type": string;
    "odata.id": string;
    "odata.editLink": string;
    ChangeToken: {
      StringValue: string;
    },
    ChangeType: ChangeType;
    SiteId: string;
    Time: Date;
    Editor: string;
    EditorEmailHint: string;
    ItemId: number;
    ListId: string;
    ServerRelativeUrl: string;
    SharedByUser: string;
    SharedWithUsers: string;
    UniqueId: string;
    WebId: string;
  }
```

Boilerplate Webhook Implementation

Subscription Registration & Queueing

Project Folder



functions	Folder that holds all of the trigger definitions for the Azure Function
eventGridSubscription.ts	Manages the subscription for the webhook to Event Grid
eventGridTrigger.ts	Trigger that runs when Event Grid needs to send a notification of a change event.

eventGridSubscription.ts

```
1   import { app, EventGridEvent, InvocationContext, output } from "@azure/functions";
2   import { IEGNotification } from "../models/SubscriptionModels.js";
3   import { AppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";
4   import { AuthService } from "../common/auth.js";
5   import "@pnp/graph/subscriptions/index.js";
6
7   const queueOutput = output.storageQueue({
8     queueName: process.env.SubscriptionQueuePath,
9     connection: 'AzureWebJobsStorage',
10  });
11
12  export async function eventGridSubscription(event: EventGridEvent, context: InvocationContext): Promise<void> {…
77  }
78
79  app.eventGrid('eventGridSubscription', {
80    handler: eventGridSubscription,
81    extraOutputs: [queueOutput]
82  });
83
```

eventGridSubscription.ts

```typescript
export async function eventGridSubscription(event: EventGridEvent, context: InvocationContext): Promise<void> {
  const LOG_SOURCE = "eventGridSubscription";
  let _apu = null;

  try {
    _apu = new AppInsightUtil();
    _apu?.Init(context.invocationId);

    _apu?.Log(MessageType.Event, {…
    });

    const expTime = new Date();
    expTime.setDate(expTime.getDate() + 29);

    const _auth = new AuthService(_apu);
    const success = _auth.Init();
    if(success){
      // Reauthorize subscription
      const subId = (event as unknown as IEGNotification).data.subscriptionId;
      const z = await _auth.graph.subscriptions.getById(subId).update({expirationDateTime: expTime.toISOString()});
      if(z.id.toLowerCase() === subId.toLowerCase()){
        _apu?.Log(MessageType.Trace, {…
        });
      }else{
        _apu?.Log(MessageType.Trace, {…
        });
        context.extraOutputs.set(queueOutput, JSON.stringify(event));
      }
    }
  } catch (err) {
    // If error then put event notification on the queue
    context.extraOutputs.set(queueOutput, JSON.stringify(event));
    if (_apu != null) {
      _apu.Log(MessageType.Exception, {…
      });
    } else {
      console.error(`${this.LOG_SOURCE} - General Failure`, err);
    }
  }
}
```

eventGridTrigger.ts

```
1   import { app, EventGridEvent, InvocationContext, output } from "@azure/functions";
2   import { AppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";
3
4   const queueOutput = output.storageQueue({
5     queueName: process.env.NotificationQueuePath,
6     connection: 'AzureWebJobsStorage',
7   });
8
9   export async function eventGridListChanged(event: EventGridEvent, context: InvocationContext): Promise<void> {···
41  }
42
43  app.eventGrid('eventGridListChanged', {
44    handler: eventGridListChanged,
45    extraOutputs: [queueOutput]
46  });
47
```

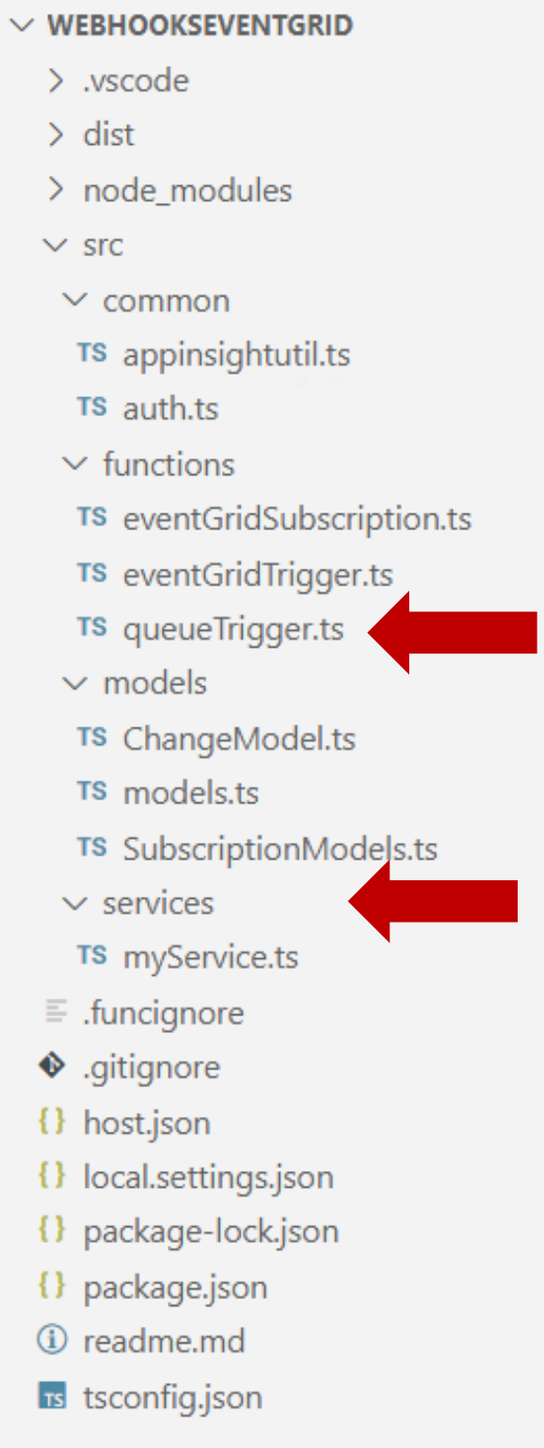
eventGridTrigger.ts

```
 9  export async function eventGridListChanged(event: EventGridEvent, context: InvocationContext): Promise<void> {
10    const LOG_SOURCE = "eventGridListChanged";
11    let _apu = null;
12
13    try {
14      _apu = new AppInsightUtil();
15      _apu?.Init(context.invocationId);
16
17  >   _apu?.Log(MessageType.Event, {···
24      });
25
26      context.extraOutputs.set(queueOutput, JSON.stringify(event));
27    } catch (err) {
28      if (_apu != null) {
29  >     _apu.Log(MessageType.Exception, {···
36        });
37      } else {
38        console.error(`${this.LOG_SOURCE} - General Failure`, err);
39      }
40    }
41  }
```

Boilerplate Webhook Implementation

Queue Processing

Project Folder



Services	Folder that holds any services created for the project
queueTrigger.ts	Processes the items added to the queue by the eventGridTrigger
myService.ts	Service that provides the business logic for how to handle changed items.

queueTrigger.ts

```
1   import { app, InvocationContext, input, output } from "@azure/functions";
2   import { IEGNotification, SubscriptionConfig } from "../models/SubscriptionModels.js";
3   import { AppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";
4   import { AuthService } from "../common/auth.js";
5   import { MyService } from "../services/myService.js";
6
7   const blobInput = input.storageBlob({
8     path: process.env.SubscriptionFile,
9     connection: 'AzureWebJobsStorage',
10  });
11
12  const blobOutput = output.storageBlob({
13    path: process.env.SubscriptionFile,
14    connection: 'AzureWebJobsStorage',
15  });
16
17  export async function notificationQueue(queueItem: IEGNotification, context: InvocationContext): Promise<void> {…
77  }
78
79  app.storageQueue('notificationQueue', {
80    queueName: process.env.NotificationQueuePath,
81    connection: 'AzureWebJobsStorage',
82    extraInputs: [blobInput],
83    extraOutputs: [blobOutput],
84    handler: notificationQueue
85  });
```

queueTrigger.ts

```typescript
export async function notificationQueue(queueItem: IEGNotification, context: InvocationContext): Promise<void> {
  const LOG_SOURCE = "notificationQueue";
  const _apu = new AppInsightUtil();
  _apu.Init(context.invocationId);

  _apu.Log(MessageType.Event, {···
  });

  try {
    const auth = new AuthService(_apu);
    const initialized = await auth.Init();
    let result = false;
    if (initialized) {
      let subscriptionConfig: SubscriptionConfig = context.extraInputs.get(blobInput) as SubscriptionConfig;
      if (subscriptionConfig == null) {
        subscriptionConfig = {
          subscriptionId: queueItem.data.subscriptionId,
          expirationDateTime: queueItem.data.subscriptionExpirationDateTime,
          lastToken: null,
          graphResource: queueItem.data.resource
        };
      }
      const lastToken = subscriptionConfig.lastToken;
      const lns = new MyService(_apu, auth);
      const updatedSubConfig = await lns.Process(subscriptionConfig);
      result = (lastToken !== updatedSubConfig.lastToken || updatedSubConfig.lastToken !== null);
      if (result) {
        context.extraOutputs.set(blobOutput, updatedSubConfig);
      }
      _apu.Log(MessageType.Trace, {···
      });
      if(!result){
        throw Error(`Failed to process queue item ${JSON.stringify(queueItem)}.`);
      }
    }
  } catch (err) {
    _apu.Log(MessageType.Exception, {···
    });
    throw Error(`Queue item was not processed. Error: ${err}`);
  }
}
```

myService.ts

```
1   import { IAppInsightUtil, MessageType, SeverityLevel } from "../common/appinsightutil.js";
2   import { IAuthService } from "../common/auth.js";
3   import { ChangeType, IChangeModel } from "../models/ChangeModel.js";
4   import { IMyItem } from "../models/models.js";
5   import { SubscriptionConfig } from "../models/SubscriptionModels.js";
6
7   import "@pnp/sp/webs";
8   import "@pnp/sp/lists";
9   import "@pnp/sp/items/index.js";
10  import "@pnp/sp/batching.js";
11  import { IChangeQuery } from "@pnp/sp/index.js";
12
13  export interface IMyService {
14    readonly ready: boolean;
15    Init: () => Promise<void>;
16    Process: (subscriptionConfig: SubscriptionConfig) => Promise<SubscriptionConfig>;
17  }
18
19  export class MyService implements IMyService {···
240 }
```

myService.ts

```
19  export class MyService implements IMyService {
20    private LOG_SOURCE = "MyService";
21    private _ready: boolean = false;
22
23    private _apu: IAppInsightUtil;
24    private _auth: IAuthService;
25
26    private _subConfig: SubscriptionConfig;
27    private _lastChangeToken: string = null;
28
29    constructor(apu: IAppInsightUtil, auth: IAuthService) {
30      this._apu = apu;
31      this._auth = auth;
32    }
33
34    public async Init(): Promise<void> {
35      this._ready = true;
36    }
37
38    public get ready(): boolean {
39      return this._ready;
40    }
41
42    public async Process(subscriptionConfig: SubscriptionConfig): Promise<SubscriptionConfig> {···
183   }
184
185   private async _getChangedItemIds(skipIds: boolean = false): Promise<number[]> {···
239   }
240 }
```

myService.ts

```
42  public async Process(subscriptionConfig: SubscriptionConfig): Promise<SubscriptionConfig> {
43    try {
44      this._subConfig = subscriptionConfig;
45      // Process the notification
46      let processingSuccess = true;
47      const changeIds = await this._getChangedItemIds();
48
49      // Process Changes Ids to get items
50  >   if (changeIds.length > 0) {···
139     } else {
140       // If no changes needed to be processed, update the last change token.
141       if (this._lastChangeToken == null) { this._lastChangeToken = this._subConfig.lastToken; }
142 >     this._apu.Log(MessageType.Trace, {···
149       });
150     }
151     /*
152     * If processing was successful and LastChangeToken isn't null
153     * then update the subscription configuration objects last change token to use next time.
154     */
155     if (processingSuccess && this._lastChangeToken != null) {
156       this._subConfig.lastToken = this._lastChangeToken;
157 >     this._apu.Log(MessageType.Trace, {···
164       });
165     } else {
166       this._subConfig.lastToken = null;
167 >     this._apu.Log(MessageType.Trace, {···
174       });
175     }
176   } catch (err) {
177     this._subConfig.lastToken = null;
178 >   this._apu.Log(MessageType.Exception, {···
185     });
186   }
187   // Return the updated subscription configuration to be saved in blob storage.
188   return this._subConfig;
189 }
```

myService.ts

```
191   private async _getChangedItemIds(skipIds: boolean = false): Promise<number[]> {
192     const retVal: number[] = [];
193     try {
194       // Process the notification
195       const changeQuery: IChangeQuery = {
196         Add: true,
197         Update: true,
198         Item: true,
199         ChangeTokenEnd: null,
200         ChangeTokenStart: null,
201       };
202
203       if (this._subConfig.lastToken != null) {
204         changeQuery.ChangeTokenStart = { StringValue: this._subConfig.lastToken };
205       }
206
207       // get list changes
208       let processingSuccess = true;
209       let items: IChangeModel[] = await this._auth.sp.web.lists.getById(process.env.ProjectListId).getChanges(changeQuery);
210       do {
211         if (items.length > 0) {
212 >         this._apu.Log(MessageType.Trace, {···
216           });
217           processingSuccess = false;
218           // Process the changes
219           // filter add and update changes
220           if (!skipIds) {
221             items.forEach(o => {
222               if (o.ChangeType === ChangeType.Add || o.ChangeType === ChangeType.Update) {
223                 if (retVal.indexOf(o.ItemId) === -1) {
224                   retVal.push(o.ItemId);
225                 }
226               }
227             });
228           }
229           this._lastChangeToken = items[items.length - 1].ChangeToken.StringValue;
230           changeQuery.ChangeTokenStart = { StringValue: this._lastChangeToken };
231           items = await this._auth.sp.web.lists.getById(process.env.ProjectListId).getChanges(changeQuery);
232         }
233       } while (items != null && items?.length > 0)
234     } catch (err) {
235 >     this._apu.Log(MessageType.Exception, {···
242       });
243     }
244     return retVal;
245   }
```

myService.ts

```
42  public async Process(subscriptionConfig: SubscriptionConfig): Promise<SubscriptionConfig> {
43    try {
44      this._subConfig = subscriptionConfig;
45      // Process the notification
46      let processingSuccess = true;
47      const changeIds = await this._getChangedItemIds();
48
49      // Process Changes Ids to get items
50      if (changeIds.length > 0) {
51        let changes: IMyItem[] = [];
52        const [batchedSP, execute] = this._auth.sp.batched();
53        for (let i = 0; i < changeIds.length; i++) {
54          batchedSP.web.lists.getById(process.env.ProjectListId).items.getById(changeIds[i]).select("Id", "Title")().then((item) => {
55            if (item != null) {
56              this._apu.Log(MessageType.Trace, {···
64              });
65              changes.push(item);
66            }
67          }, () => {
68            this._apu.Log(MessageType.Trace, {···
75            });
76          });
77        }
78        await execute();
79
80        this._apu.Log(MessageType.Trace, {···
88        });
89
90        // Process changed items
91        if (changes.length > 0) {···
136       } else {···
138       }
139     } else {···
150     }
151     /*
152     * If processing was successful and LastChangeToken isn't null
153     * then update the subscription configuration objects last change token to use next time.
154     */
155     if (processingSuccess && this._lastChangeToken != null) {···
165     } else {···
175     }
176   } catch (err) {···
186   }
187   // Return the updated subscription configuration to be saved in blob storage.
188   return this._subConfig;
189 }
```

myService.ts

```
49      // Process Changes Ids to get items
50      if (changeIds.length > 0) {
51        let changes: IMyItem[] = [];
52        const [batchedSP, execute] = this._auth.sp.batched();
53 >      for (let i = 0; i < changeIds.length; i++) {···
77        }
78        await execute();
79
80 >      this._apu.Log(MessageType.Trace, {···
88        });
89
90        // Process changed items
91        if (changes.length > 0) {
92 >        this._apu.Log(MessageType.Trace, {···
100       });
101       processingSuccess = false;
102       let processedCount = 0;
103       for (let c = 0; c < changes.length; c++) {
104         try {
105           /*
106           *
107           *  DO SOMETHING WITH THE CHANGED ITEM
108           *
109           */
110           processedCount++;
111         } catch (err) {
112 >         this._apu.Log(MessageType.Exception, {···
120         });
121         }
122       }
123       processingSuccess = processedCount === changes.length;
124       if (!processingSuccess) {
125 >       this._apu.Log(MessageType.Trace, {···
134       });
135       }
136     } else {
137       processingSuccess = true;
138     }
139 >   } else {···
150     }
```

Summary

- Set up the Azure Infrastructure
- Created a NodeJS project and configured it
- Added helper functions for logging and authentication
- Created TypeScript Interfaces for SharePoint list changes and EventGrid subscriptions
- Created triggers to handle subscription renewal and events, putting the events on a queue to be processed
- Processed the change events and stored the last change token so that we only pick up changes that we haven't successfully processed when a new event fires.

Resources

- Overview of SharePoint webhooks
- Microsoft Graph Subscription Resource
- Microsoft Graph API change notifications
- Subscribe to events published by a partner with Azure Event Grid
- Receive Microsoft Graph API change events through Azure Event Grid
- How to find SharePoint site collections and OneDrive for Business drives
- Setting Up Azure Event Grid